ACE ENGINEERING COLLEGE

ONE-WEEK TECHNICAL PROJECT REPORT

Project Title: Astra AI — Educational Password-Security Analysis and Hashing Guidance

Submitted by: [TO BE FILLED BY STUDENT]

Roll Number(s): [TO BE FILLED BY STUDENT]

Branch and Year: [TO BE FILLED BY STUDENT]

Academic Year: [TO BE FILLED BY STUDENT]

Project Duration: [Start Date] to [End Date] (7 Days; actual dates to be filled)

Project Guide / Mentor: [TO BE FILLED BY STUDENT]

Organization / Training Institute: [TO BE FILLED BY STUDENT, if applicable]

Submitted to the Department of [TO BE FILLED BY STUDENT]

TABLE OF CONTENTS

1. Certificate

2. Acknowledgement

3. Abstract

4. Introduction

5. Objectives of the Project

6. Problem Statement and Scope

7. Tools and Technologies Used

8. System Design / Methodology

9. Day-Wise Progress Report

10. Implementation and Results

11. Learning Outcomes

12. Challenges Faced and Solutions

13. Conclusion and Future Scope

14. References

15. Appendices / Supporting Evidence

16. Mentor Evaluation Sheet

1. CERTIFICATE (IF APPLICABLE)

This is to certify that [TO BE FILLED BY STUDENT], bearing Roll Number(s) [TO BE FILLED BY STUDENT], of [Department and Year], has/have completed the one-week technical project entitled "Astra AI — Educational Password-Security Analysis and Hashing Guidance" during [Start Date] to [End Date] at [Organization / Institution, if applicable], under the guidance of [Mentor Name and Designation]. This certificate is subject to institutional approval and remains unsigned.

Signature of Project Mentor: _____

Signature of Department Project Guide: _____

Date: _____ Place: _____

2. ACKNOWLEDGEMENT

I express my gratitude to [Project Guide / Mentor] and the faculty of [Department Name], ACE Engineering College, for their guidance and support during this project. I also thank [Organization / Training Institute, if applicable] and everyone who provided technical feedback or helped review the application. The student should complete or adjust these acknowledgements to reflect the support actually received.

3. ABSTRACT

Astra AI is a Java-based educational desktop application for local password-characteristic analysis and guidance about password-storage algorithms. The project presents a three-state workflow—PASSWORD_ENTRY, PROCESSING and FINAL_RESULT—and assesses derived characteristics such as length, character classes and predictable patterns. Its report covers Argon2id, bcrypt, scrypt, PBKDF2 and a SHA-3-based entry that remains not estimable because no concrete password KDF is specified. Four illustrative crack-time estimates use the project-defined educational-reference-v1 assumptions; these values are neither measurements nor predictions of real attacks, true entropy or attacker ordering. A separate deterministic simulation coordinates 100 logical agents through eight bounded workers over synthetic candidate indexes; it does not receive, hash or crack the submitted password, and its throughput is unrelated to the educational estimates. An optional local KDF verification benchmark uses a random fixture and is separate from both the password analysis and estimator. The application is implemented with Java 21 and JavaFX, built with Maven and tested with JUnit. In this validation run, 134 automated tests passed with no failures, errors or skips, and Maven packaging succeeded. The project demonstrates how to explain password-related concepts while limiting claims about memory handling, estimation and benchmark evidence.

4. INTRODUCTION

Passwords are often converted into stored verifiers with password-based key derivation functions (KDFs) rather than used directly as cryptographic keys. KDF design and configuration affect verification cost, while salts and deployment requirements matter to secure storage. Argon2id, bcrypt, scrypt and PBKDF2 are password-oriented mechanisms with tunable costs; SHA-3 is a hash-function family and a bare digest is not, by itself, a complete password-storage KDF.

Astra AI was developed as a teaching aid that makes local heuristic feedback and algorithm guidance visible in a desktop workflow. It is not an authentication service, password cracker or substitute for a security review. Its estimator uses simplified assumptions authored for this project and presents educational comparisons only.

5. OBJECTIVES OF THE PROJECT

1. Build a JavaFX workflow with PASSWORD_ENTRY, PROCESSING and FINAL_RESULT states.
2. Analyze character classes and predictable patterns locally, keeping derived counts in the analysis record.
3. Present guidance for Argon2id, bcrypt, scrypt, PBKDF2 and an underspecified SHA-3-based entry.
4. Keep synthetic-index simulation separate from the submitted password.
5. Display internal educational assumptions and their limitations.
6. Verify behaviors with tests and Maven builds.

6. PROBLEM STATEMENT AND SCOPE

Problem statement: Password-security concepts can be difficult to compare when algorithm names are shown without context, and simple strength labels can be mistaken for guarantees. The project addresses this educational gap with a local heuristic, scoped algorithm guidance and an explicitly limited illustrative estimator.

Scope: Included are a JavaFX entry/processing/results flow, local character-based analysis, five algorithm entries, four assumption-based estimate profiles, a separate synthetic candidate-index simulation, an opt-in local KDF verification benchmark using a random fixture, JSON report validation and automated tests. The application does not transmit or persist the submitted password, perform authentication, test external systems, crack or benchmark the submitted password, or predict an attacker's duration. The estimates do not represent measured hardware performance or true password entropy.

7. TOOLS AND TECHNOLOGIES USED

Tool / Technology	Purpose	Version / Details
Java Development Kit	Language/runtime for application and tests	OpenJDK Temurin 21.0.12.1; pom targets Java 21
JavaFX	Desktop controls and presentation states	21.0.6 (javafx-controls)
Apache Maven	Dependency management, tests and packaging	3.10.0 in this validation environment; compiler plugin 3.13.0, Surefire 3.5.2
JUnit Jupiter	Automated unit and view tests	5.11.4
Bouncy Castle	Local Argon2id, bcrypt and scrypt benchmark implementations	bcprov-jdk18on 1.86
Git	Working-tree and whitespace validation	Version not recorded in this run
Windows 11	Development/build environment	10.0; amd64 (Maven environment report)

8. SYSTEM DESIGN / METHODOLOGY

The JavaFX application maps its main screens to PASSWORD_ENTRY, PROCESSING and FINAL_RESULT. On submission, the visible/password text is read transiently, converted to a mutable character array for the local heuristic, and the fields are cleared. The analysis record contains derived

counts and explanations, not the submitted text. The mutable char[] working copy is cleared by the input policy where implemented. JavaFX getText() creates an immutable Java String, which cannot be reliably wiped; therefore the application does not claim complete in-memory erasure. The UI fields are cleared after submission, and the password is not persisted or transmitted. The heuristic is limited: it is not a breach corpus, probabilistic password model or cryptographic entropy measurement.

Separately, the processing lifecycle coordinates 100 logical agent/state objects over a bounded worker pool of eight threads. The default challenge partitions 100,000 deterministic candidate indexes into 100 tasks and has a synthetic match at index 4,271. The challenge consumes indexes only. Its result and throughput do not test the submitted password and are not inputs to the crack-time estimator.

The final screen presents local analysis and five algorithm cards. Four cards use educational-reference-v1: Argon2id v1.3, m=65,536 KiB, t=3, p=1 at 10 guesses/s; bcrypt 2b, cost=12 at 10 guesses/s; scrypt N=131,072, r=8, p=1 at 5 guesses/s; and PBKDF2-HMAC-SHA-256, 600,000 iterations at 50 guesses/s. These are internal illustrative assumptions, not measured or externally sourced rates. The estimator applies a simplified candidate-space model and expected-work convention; outputs are not predictions, true entropy or actual attacker ordering. SHA-3 remains not estimable because no concrete password KDF is specified.

The optional KDF verification benchmark is a separate developer tool that creates a random fixture, derives and verifies it locally and can write a JSON observation. Verification latency is not cracking throughput. Reports are unsigned: matching configuration/runtime metadata is a consistency check, not cryptographic authenticity or tamper resistance.

Password path: [Entry field] → [Local heuristic] → [Derived counts] → [Report / estimate]
Separate simulation: [PROCESSING] → [100 logical agents / 8 workers] → [Synthetic candidate indexes]

Figure : Text workflow showing the local password-analysis path and independent synthetic-index simulation.

9. DAY-WISE PROGRESS REPORT

Day-to-activity mapping below is proposed from the available project and conversation evidence; confirm the actual dates and order before submission.

Day	Activities Performed	Deliverables / Outcomes
Day 1 (proposed; confirm)	Review project brief, repository and educational boundaries.	Scope, states and verification targets recorded.
Day 2 (proposed; confirm)	Inspect JavaFX entry workflow and refine transient local heuristic analysis.	PasswordAnalysis/input-policy path; fields clear on submit.
Day 3 (proposed; confirm)	Develop algorithm catalog and educational guidance for five entries.	Argon2id, bcrypt, scrypt, PBKDF2 and underspecified SHA-3 guidance.

Day 4 (proposed; confirm)	Implement the separate deterministic synthetic-index simulation lifecycle.	100 logical agents, 100 tasks, 100,000 indexes; bounded eight-worker execution.
Day 5 (proposed; confirm)	Implement project-defined crack-time assumptions and estimator with explicit limits.	Four reference profiles; SHA-3 not estimable; estimates isolated from throughput.
Day 6 (proposed; confirm)	Integrate opt-in KDF verification benchmark/report validation and refine Results view.	Random fixture, unsigned JSON observation, compact cards and view tests.
Day 7 (proposed; confirm)	Correct misleading terminology, review regression coverage and validate the build.	Logical-agent wording, banner removal, 134 passing tests and successful package.

10. IMPLEMENTATION AND RESULTS

Implementation is organized around the application entry point (com.astraai.Main), application shell/state mapping, password-entry policy and immutable analysis record, algorithm catalog/guidance and result cards, crack-time dataset/estimator, and the separate simulation engine. The optional KDF verification benchmark and report loader are standalone from the JavaFX password-analysis path. In a graphical desktop session, the configured launch command is mvn javafx:run; the Maven plugin entry point is com.astraai.Main.

Automated tests cover simulation configuration/lifecycle, input policy, algorithm catalog and guidance, estimate assumptions and formatting, benchmark configuration/statistics/report validation, state mapping and Results-view structure. Current validation with OpenJDK Temurin 21.0.12.1 and Maven 3.10.0: mvn test completed with 134 tests, 0 failures, 0 errors and 0 skipped; mvn package completed with all 134 tests passing and produced a successful package. git diff --check exited successfully; Git emitted line-ending normalization notices for existing edited source files.

Results are evidenced by source inspection and automated tests. The JavaFX application was not launched for manual visual observation in this report run, so no claim of runtime visual verification is made. No KDF performance benchmark was run. Crack-time values are model outputs under project-defined assumptions, not experimental results.

Result / Observation: Source and automated tests support the three-state flow, local derived-characteristic analysis, separate synthetic simulation, five algorithm cards, four illustrative estimate profiles and optional benchmark/report components. No submitted-password cracking, external network activity, manual JavaFX screenshot or performance-benchmark observation is claimed.

11. LEARNING OUTCOMES

The code and tests provide evidence of work with Java records, immutable data, validation, JavaFX controls/layouts, state mapping, bounded concurrency, deterministic simulation, JUnit assertions and Maven lifecycle commands. The project also distinguishes password KDF verification from guessing-

rate estimates and separates measured fixture observations from internally authored illustrative assumptions. Student-specific learning reflections should be added by the student.

12. CHALLENGES FACED AND SOLUTIONS

Challenge	Action / Solution	Outcome
Results algorithm cards could exceed the available width.	Refined the existing compact responsive card layout and added focused view assertions.	Regression tests cover view structure; manual visual review remains outstanding.
Application identity regression returned an incomplete display name.	Traced and corrected the production metadata source while retaining expected-identity coverage.	The full test suite currently passes, including the metadata test.
Processing terminology could imply 100 real AI models.	Changed wording to “Preparing 100 logical simulation agents” and added an assertion.	Text reflects logical objects; worker count and simulation behavior remain unchanged.
A synthetic simulation result distracted from password Results.	Removed its result panel from the Results layout while retaining simulation engine and lifecycle.	View test asserts banner absence; no manual visual inspection is claimed.
Educational estimates could be mistaken for measured cracking predictions.	Labeled rates internal and illustrative; separated estimates from benchmark/synthetic throughput.	Source/test evidence supports separation; assumptions remain unmeasured.

13. CONCLUSION AND FUTURE SCOPE

Conclusion: The project implements an educational password-security analysis workflow with local heuristic feedback, algorithm guidance and clearly qualified estimate outputs. It separates the submitted password from a synthetic-index simulation and from an opt-in random-fixture KDF benchmark. Source and automated checks support these functional boundaries. Final academic acceptance and the accuracy of personal details and actual dates remain for the student and institution to confirm.

Future scope: Possible work includes accessibility and manual visual review across supported screen sizes, student-approved screenshots, a probability-aware password-strength model with carefully sourced evaluation evidence, configuration-aware KDF guidance reviewed against current authoritative recommendations, and signed benchmark-report provenance if authenticity guarantees are required. Extensions should retain the separation between verification-cost measurements and attacker guessing estimates.

14. REFERENCES

References consulted (accessed 21 October 2026):
[1] OpenJDK, Java SE 21 Documentation, https://docs.oracle.com/en/java/javase/21/
[2] OpenJFX, Getting Started with JavaFX, https://openjfx.io/openjfx-docs/index.html
[3] Apache Maven Project, Maven Documentation and Build Lifecycle, https://maven.apache.org/guides/
[4] B. Biryukov, D. Dinu and D. Khovratovich, RFC 9106: Argon2 Memory-Hard Function for

Password Hashing and Proof-of-Work Applications, IETF, 2021, https://www.rfc-editor.org/rfc/rfc9106.html
[5] C. Percival and S. Josefsson, RFC 7914: The scrypt Password-Based Key Derivation Function, IETF, 2016, https://www.rfc-editor.org/rfc/rfc7914.html
[6] K. Moriarty, B. Kaliski and A. Rusch, RFC 8018: PKCS #5: Password-Based Cryptography Specification Version 2.1, IETF, 2017, https://www.rfc-editor.org/rfc/rfc8018.html
[7] NIST, FIPS 202: SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions, 2015, https://csrc.nist.gov/pubs/fips/202/final
[8] NIST, SP 800-132: Recommendation for Password-Based Key Derivation, Part 1: Storage Applications, 2010, https://csrc.nist.gov/pubs/sp/800/132/final
[9] N. Provos and D. Mazières, A Future-Adaptable Password Scheme, USENIX Annual Technical Conference, 1999, https://www.usenix.org/conference/1999-usenix-annual-technical-conference/future-adaptable-password-scheme
[10] Project source and tests: pom.xml, README.md, src/main/java/com/astraai, src/main/java/com/astraai/ui, src/main/java/com/astraai/simulation and src/test/java. These are implementation evidence; project-defined crack-time assumptions in CrackTimeAssumptionDataset are not external reference values.

15. APPENDICES / SUPPORTING EVIDENCE

Available supporting evidence: project source files and automated test reports. This run did not produce or verify a JavaFX screenshot, and no screenshot attachment was available to insert. Add approved screenshots here after manual visual verification. Personal/institutional documents, attendance proof and certificates are not included unless supplied by the student.

16. MENTOR EVALUATION SHEET (SUGGESTED)

Student Name(s): _____

Project Title: Astra AI — Educational Password-Security Analysis and Hashing Guidance

Mentor Name: _____

Evaluation Criterion	Maximum Marks	Marks Awarded
Understanding of problem and objectives	10	
Technical knowledge and methodology	20	
Implementation / practical work	25	
Results and analysis	15	
Report quality and documentation	15	
Presentation and viva-voce	15	
Total	**100**	

Mentor Remarks: _____

Mentor Signature: _____ Date: _____ HoD Signature: _____